Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 8 – Apply Polymorphism 4 Marks

Bloom's Level: K4 – Analyse

Demonstrate both compile-time and runtime polymorphism in your design.

Discuss:

- At least one overloaded method (for example: calculateFee(int minutes) and calculateFee(LocalTime in, LocalTime out)) and how the compiler selects it
- How a single array of Vehicle references can hold different vehicle types (upcasting)
- How dynamic method dispatch selects the correct calculateFee() at runtime
- Why this design avoids long if–else chains that test the vehicle type

Trace one call through **dynamic method dispatch**, showing the declared type, the actual object type and the method executed.

Complete the Code

Complete the Vehicle hierarchy below and execute FeeDemo. It applies inheritance (Task 6), overriding (Task 7), polymorphism (Task 8) and an interface (Task 9).

```
interface Chargeable {
   double chargingFee(int kWh); }

abstract class Vehicle {    private final String regNo;    protected Vehicle(String regNo)

{ this.regNo = regNo; }    public String getRegNo() { return regNo; }    protected

abstract double ratePerHour();

   /** Every started hour is charged in full. */    public double calculateFee(int
minutes) {
      // TODO 1: Compute billable hours as the ceiling of minutes / 60 using integer
      //   arithmetic only, and return hours × ratePerHour()


   }

   @Override public boolean equals(Object o) {
      return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);    }

   @Override public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
   TwoWheeler(String regNo) { super(regNo); }
   @Override protected double ratePerHour() { return 10; }
}

class Car extends Vehicle {
   // TODO 2: Write the constructor using super, and override ratePerHour() to return
   //   30


}

class ElectricCar extends Car implements Chargeable {
   ElectricCar(String regNo) { super(regNo); }
   // TODO 3: Implement chargingFee() at Rs 12 per kWh


}

public class FeeDemo {
   public static void main(String[] args) {
      Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"),                    new
ElectricCar("TN03C3") };        for (Vehicle v : parked) {
         double fee = v.calculateFee(130);         // dynamic method dispatch            if (v instanceof Chargeable c) fee +=
c.chargingFee(10);
         System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo()
               + " -> Rs " + fee);
      }
      System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
   }
}
```

Expected output:

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0 true
```

Explain why the last line prints true, and decide whether this is the correct behaviour for your system.

Alternative Concept and Complexity Analysis

You used dynamic method dispatch. If the same behaviour were written using instanceof checks and casting for each vehicle type, what would the solution be? Analyse the time per call for k vehicle types (O(1) dispatch versus up to k checks) and the effect on code size and maintenance.

Evidence Expected

Working examples of overloading and dynamic dispatch with a traced call and an explanation of the design benefit.

Solution:

TASK 8 – Apply Polymorphism

Campus Smart-Parking System – 4 Marks

1. Introduction to Polymorphism

Polymorphism means one method or reference can represent different behaviours depending on the situation.

Java supports two types of polymorphism:

1. Compile-time polymorphism: Achieved through method overloading.
2. Runtime polymorphism: Achieved through method overriding and dynamic method dispatch.

2. Compile-Time Polymorphism – Method Overloading

Method overloading means defining multiple methods with the same name but different parameter lists.

Example:

double calculateFee(int minutes)

Calculates the fee using parking duration in minutes.

double calculateFee(LocalTime in, LocalTime out)

Calculates the fee using entry and exit times.

The compiler selects the correct method based on the arguments supplied. This is called compile-time polymorphism.

3. Runtime Polymorphism – Dynamic Method Dispatch

A superclass reference can hold objects of different subclasses through upcasting.

Example:

```
Vehicle[] parked = {
   new TwoWheeler("TN01A1"),
   new Car("TN02B2"),
```

```
    new ElectricCar("TN03C3")
};
```

Although all references are declared as Vehicle, each object executes its own overridden ratePerHour() method.

This is called dynamic method dispatch.

Dynamic Method Dispatch Trace

Consider:

```
Vehicle v = new ElectricCar("TN03C3");
double fee = v.calculateFee(130);
```

Stage	Description
Declared type	Vehicle
Actual object type	ElectricCar
Called method	calculateFee(130)
Rate method executed	ElectricCar inherits Car's ratePerHour()
Calculation	3 × 30 = Rs.90

Every started hour is charged in full. Therefore, 130 minutes is charged as 3 hours.

4. Advantages of Polymorphism

- A single Vehicle array can store different vehicle objects.
- Each subclass provides its own behaviour.
- It avoids lengthy if–else statements for checking vehicle types.
- New vehicle types can be added with less modification to existing code.
- It improves code reusability and maintainability.

5. Complete Java Program

File name: FeeDemo.java

```
import java.time.LocalTime;
import java.time.Duration;


interface Chargeable {
    double chargingFee(int kWh);
```

```
}
abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) {
        this.regNo = regNo;
    }
    public String getRegNo() {
        return regNo;
    }
    protected abstract double ratePerHour();
    // Overloaded method 1: Calculate fee using minutes
    public double calculateFee(int minutes) {
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }
    // Overloaded method 2: Calculate fee using entry and exit times
    public double calculateFee(LocalTime in, LocalTime out) {
        long minutes = Duration.between(in, out).toMinutes();
        if (minutes < 0) {
            minutes += 24 * 60;
        }

        return calculateFee((int) minutes);
    }

    @Override
    public boolean equals(Object o) {
        return o instanceof Vehicle
```

```
            && regNo.equals(((Vehicle) o).regNo);
    }

    @Override
    public int hashCode() {
        return regNo.hashCode();
    }
}
class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) {
        super(regNo);
    }
    @Override
    protected double ratePerHour() {
        return 10;
    }
}
class Car extends Vehicle {
    Car(String regNo) {
        super(regNo);
    }
    @Override
    protected double ratePerHour() {
        return 30;
    }
}
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) {
```

```
        super(regNo);
    }
    @Override
    public double chargingFee(int kWh) {
        return kWh * 12;
    }
}
public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = {
            new TwoWheeler("TN01A1"),
            new Car("TN02B2"),
            new ElectricCar("TN03C3")
        };
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);
            // Runtime checking for charging capability
            if (v instanceof Chargeable c) {
                fee += c.chargingFee(10);
            }
            System.out.println(
                v.getClass().getSimpleName() + " "
                + v.getRegNo() + " -> Rs " + fee
            );
        }
        System.out.println(
            new Car("TN02B2").equals(
                new ElectricCar("TN02B2")
```

```
            )
        );

        // Demonstrating overloaded method
        Vehicle car = new Car("TN04D4");
        System.out.println(
            "Time-based fee: Rs " +
            car.calculateFee(
                LocalTime.of(9, 0),
                LocalTime.of(11, 10)
            )
        );
    }
}
```

6. Expected Output

TwoWheeler TN01A1 -> Rs 30.0

Car TN02B2 -> Rs 90.0

ElectricCar TN03C3 -> Rs 210.0

true

Time-based fee: Rs 90.0

Fee Calculation

Vehicle	Hourly rate	Parking fee (3 hours)	Charging fee	Total
TwoWheeler	Rs.10	Rs.30	–	Rs.30
Car	Rs.30	Rs.90	–	Rs.90
ElectricCar	Rs.30	Rs.90	Rs.120	Rs.210

Why does the last equality statement print true?

```
new Car("TN02B2").equals(
    new ElectricCar("TN02B2")
```

)

It prints true because the equals() method compares only the registration number. Both objects have the same registration number, TN02B2.

Is this correct?

Yes, if the system considers registration numbers globally unique identifiers for vehicles, regardless of vehicle category. The hashCode() method also uses the registration number, maintaining the equality contract.

If the system needs to distinguish vehicle categories even when registration numbers match, the equality design should be changed.

7. Alternative Concept – instanceof Checks and Casting

Instead of dynamic method dispatch, the fee calculation could use instanceof checks.

Example:

```
double calculateFee(Vehicle v, int minutes) {
   int hours = (minutes + 59) / 60;
   double rate;
   if (v instanceof ElectricCar) {
      rate = 30;
   } else if (v instanceof Car) {
      rate = 30;
   } else if (v instanceof TwoWheeler) {
      rate = 10;
   } else {
      rate = 0;
   }
   return hours * rate;
}
```

Comparison

Factor	**Dynamic method dispatch**	**instanceof checks**
Method selection	JVM selects overridden method	Program checks object type
Time complexity	O(1)	Up to O(k) checks

Code size	Less repeated code	More conditions
Adding new vehicle	Add a subclass	Modify checking logic
Maintenance	Easier	More difficult as types increase

Here, k represents the number of vehicle types.

Conclusion: Dynamic method dispatch is more suitable for the Campus Smart-Parking System because it allows different vehicle objects to execute their own methods through a common Vehicle reference. Method overloading provides compile-time polymorphism, while overriding provides runtime polymorphism.

